

```
In [ ]: import marimo as mo
import pandas as pd
import time
from pathlib import Path
from ortools.sat.python import cp_model

# Dias e tempos letivos fixos da semana (usados em todas as secções)
DIAS = ["Seg", "Ter", "Qua", "Qui", "Sex"]
PERIODOS = [1, 2, 3, 4, 5]
```

Gerador de Horário Escolar

Notebook principal do trabalho prático. Vai sendo construído secção a secção sobre o enunciado (`horario_escolar_enunciado.py`).

Uso de ferramentas LLM

Este notebook foi construído com apoio de um LLM (Claude, Anthropic). Por convenção neste notebook:

- todas as células de código geradas ou fortemente assistidas pelo LLM têm um comentário `# [LLM]` no topo, seguido de uma nota sobre o que foi gerado e o que foi revisto/alterado manualmente;
- **LINK DO DIÁLOGO COM O LLM:** <https://claude.ai/share/7c15052e-14cb-4e21-8452-5dc64ddda88e> — incluído também no relatório entregue, como pede o enunciado da disciplina (o diálogo conta como "código complementar" para efeitos de avaliação).

1. Dados de entrada

Os dados nunca estão escritos no código (R8): são sempre lidos de uma pasta com 4 ficheiros CSV (`turmas.csv` , `disciplinas.csv` , `salas.csv` , `disponibilidade_excecoes.csv`). A classe `Dados` guarda tudo já num formato conveniente para o modelo:

- `turmas` : lista de nomes de turma;
- `disciplinas` : DataFrame com `duplo_periodo` já convertido para `bool` e `sala_especial` normalizada (`"\"` quando não se aplica);
- `capacidade_normal` : nº total de salas normais disponíveis em simultâneo (soma de todas as linhas `tipo == "normal"` , caso haja mais do que uma);
- `capacidade_especial` : dicionário `{nome_da_sala: quantidade}` só para as salas do `tipo == "especial"` — a chave corresponde diretamente ao

valor usado em `sala_especial`;

- `indisponibilidades`: conjunto de tuplos `(professor, dia, periodo)` em que esse professor **não** está disponível; por omissão (fora deste conjunto) um professor está sempre disponível, como diz o enunciado.

Se a pasta trocar de dados (mais turmas, mais disciplinas, mais exceções), o código continua a funcionar sem alterações.

```
In [ ]: # [LLM] Estrutura de dados (classe Dados) gerada com apoio do LLM;
# revista para garantir que os nomes dos campos refletem exatamente
# o vocabulário do enunciado (turmas, capacidade_normal, etc.).
class Dados:
    """Estrutura com os dados de entrada já normalizados."""

    def __init__(self, turmas, disciplinas, capacidade_normal,
                  capacidade_especial, indisponibilidades):
        self.turmas = turmas
        self.disciplinas = disciplinas
        self.capacidade_normal = capacidade_normal
        self.capacidade_especial = capacidade_especial
        self.indisponibilidades = indisponibilidades

    def __repr__(self):
        return (
            f"Dados(turmas={self.turmas}, "
            f"disciplinas={len(self.disciplinas)}, "
            f"cap_normal={self.capacidade_normal}, "
            f"cap_especial={self.capacidade_especial}, "
            f"indisponibilidades={len(self.indisponibilidades)})"
        )
```

```
In [ ]: # [LLM] Função de leitura/normalização dos CSVs gerada com apoio do
# LLM e validada manualmente contra os ficheiros reais de dados/ e
# dados_v2/ (ver diálogo linkado acima) antes de ser aceite.
def carregar_dados(pasta) -> Dados:
    """Lê os 4 CSVs de `pasta` e devolve um objeto Dados normalizado."""
    pasta = Path(pasta)

    turmas = pd.read_csv(pasta / "turmas.csv")
    ["turma"].astype(str).tolist()

    disciplinas = pd.read_csv(pasta / "disciplinas.csv")
    disciplinas["sala_especial"] = (
        disciplinas["sala_especial"].fillna("").astype(str).str.strip()
    )
    disciplinas["duplo_periodo"] = (
        disciplinas["duplo_periodo"].astype(str).str.strip().str.lower() ==
        "sim"
    )
    disciplinas["carga_semanal"] = disciplinas["carga_semanal"].astype(int)

    salas = pd.read_csv(pasta / "salas.csv")
    capacidade_normal = int(
        salas.loc[salas["tipo"] == "normal", "quantidade"].sum()
```

```

)
especiais = salas[salas["tipo"] == "especial"]
capacidade_especial = dict(
    zip(especiais["sala"], especiais["quantidade"].astype(int))
)

exc_path = pasta / "disponibilidade_excecoes.csv"
if exc_path.exists():
    exc = pd.read_csv(exc_path)
    indisponibilidades = set(
        zip(exc["professor"], exc["dia"], exc["periodo"].astype(int))
    )
else:
    indisponibilidades = set()

return Dados(
    turmas, disciplinas, capacidade_normal,
    capacidade_especial, indisponibilidades,
)

```

Explorar os dados carregados

`dados_extra/` é um terceiro conjunto de dados (não fornecido no enunciado) que acrescenta uma turma (7ºC) e duas disciplinas (Geografia, Artes — esta última também de duplo período e a usar o *mesmo* tipo de sala especial que Ciências, para testar bem a capacidade de sala). Serve para confirmar que nada no código está "hardcoded" para os dados originais (critério pedido em "Como testar/validar").

```

In [ ]: seletor_pasta = mo.ui.dropdown(
        options=["dados", "dados_v2", "dados_extra"], value="dados",
        label="Pasta de dados"
    )
seletor_pasta

```

Pasta de dados

```

In [ ]: dados = carregar_dados(seletor_pasta.value)
dados

```

Dados(turmas=['7ºA', '7ºB'], disciplinas=6, cap_normal=6, cap_especial={'Laboratório': 1, 'Ginásio': 1}, indisponibilidades=15)

```

In [ ]: mo.vstack([
    mo.md(f"**Turmas:** {', '.join(dados.turmas)}"),
    mo.ui.table(dados.disciplinas, label="Disciplinas"),
    mo.md(
        f"**Capacidade salas normais:** {dados.capacidade_normal} \n"
        f"**Capacidade salas especiais:** {dados.capacidade_especial} \n"
        f"**Nº de exceções de indisponibilidade:**"
        {len(dados.indisponibilidades)}")

```

```
) ,  
])
```

Turmas: 7ºA, 7ºB

Disciplinas

Q Search...					
☐	disciplina str	professor str	carga_semanal int64	duplo_periodo bool	sala_especial str
☐	Matemática	Prof. Ana		4 False	
☐	Português	Prof. Bruno		4 False	
☐	Ciências	Prof. Carla		3 False	Laboratório
☐	História	Prof. Diana		2 False	
☐	Inglês	Prof. Diana		2 False	
☐	Educação Física	Prof. Eduardo		2 True	Ginásio
6 rows, 5 columns					No selection

Capacidade salas normais: 6

Capacidade salas especiais: {'Laboratório': 1, 'Ginásio': 1}

Nº de exceções de indisponibilidade: 15

2. Modelo (CP-SAT / OR-Tools)

Escolha de técnica: CSP com *Constraint Programming* via OR-Tools CP-SAT (sugestão da disciplina). Justificação: o problema é essencialmente combinatório (variáveis discretas, restrições lógicas/de capacidade), o CP-SAT lida bem com isso à escala do enunciado, suporta *hints* (úteis na secção 3, construção incremental) e o objetivo O1 é uma função linear simples de somar.

Variável de decisão: para cada (turma, disciplina, dia, período de início) existe uma variável booleana `y` que vale 1 se essa disciplina começa a ser dada a essa turma nesse dia/período. Para disciplinas de duplo período, o "período de início" `p` representa um bloco que ocupa os períodos `p` e `p+1`.

Só se criam variáveis `y` para combinações onde o professor está disponível em todos os períodos do bloco — isto implementa **R6** sem precisar de uma restrição extra (a variável simplesmente não existe se violar a disponibilidade).

A partir de `y` define-se `ocupa(turma, disciplina, dia, período)` — uma expressão (não uma variável nova) que soma as variáveis `y` relevantes (a que começa nesse período, mais a do período anterior se for a segunda metade de um bloco duplo) — e usa-se essa expressão em todas as restantes restrições (R1, R5, R7), evitando duplicar lógica.

```
In [ ]: # [LLM] Gerado com apoio do LLM; validado manualmente com os dados  
# reais de dados/ (ver célula de exploração da secção 1 e o diálogo  
# linkado no topo do notebook).  
def n_ocorrencias(row):  
    """Nº de ocorrências (aulas ou blocos duplos) por semana."""  
    if row.duplo_periodo:  
        if row.carga_semanal % 2 != 0:  
            raise ValueError(  
                f"{row.disciplina}: carga_semanal ímpar com
```

```

duplo_periodo=sim"
    )
    return row.carga_semanal // 2
    return row.carga_semanal

```

```

In [ ]: # [LLM] Gerado com apoio do LLM; validado manualmente (ver secção 1).
def inicios_permitidos(row, dia, indisponibilidades):
    """Períodos de início possíveis para (row, dia), dada a
    disponibilidade do professor (implementa R6 por construção)."""
    prof = row.professor
    if row.duplo_periodo:
        candidatos = [p for p in PERIODOS if p + 1 in PERIODOS]
        return [
            p for p in candidatos
            if (prof, dia, p) not in indisponibilidades
            and (prof, dia, p + 1) not in indisponibilidades
        ]
    return [p for p in PERIODOS if (prof, dia, p) not in
    indisponibilidades]

```

Construção do modelo (R1, R2, R3, R4, R5, R7)

```

In [ ]: # [LLM] Núcleo do modelo CP-SAT gerado com apoio do LLM. Não foi
# possível correr o OR-Tools no ambiente onde este notebook foi
# inicialmente redigido (sem acesso à biblioteca); a lógica foi
# validada manualmente e por inspeção, e corrigida iterativamente
# com base nos resultados de execução reais (ver diálogo linkado).
def construir_modelo(dados):
    model = cp_model.CpModel()
    disc = dados.disciplinas

    # y[(turma, idx_disciplina, dia, periodo_inicio)] = 1 se a aula
    # começa nesse dia/período.
    y = {}
    for t in dados.turmas:
        for idx, row in disc.iterrows():
            for dia in DIAS:
                for p in inicios_permitidos(row, dia,
                dados.indisponibilidades):
                    y[(t, idx, dia, p)] =
    model.NewBoolVar(f"y_{t}_{idx}_{dia}_{p}")

    def ocupa(t, idx, dia, p):
        """Expressão booleana: turma t tem a disciplina idx a decorrer
        no período p desse dia (conta o início e, se for duplo
        período, também a segunda metade de um bloco iniciado em
        p-1)."""
        row = disc.loc[idx]
        termos = []
        if (t, idx, dia, p) in y:
            termos.append(y[(t, idx, dia, p)])
        if row.duplo_periodo and (t, idx, dia, p - 1) in y:
            termos.append(y[(t, idx, dia, p - 1)])
        return sum(termos) if termos else 0

```

```

# R2: carga semanal exata (nº de ocorrências).
for t in dados.turmas:
    for idx, row in disc.iterrows():
        vars_disc = [y[k] for k in y if k[0] == t and k[1] == idx]
        model.Add(sum(vars_disc) == n_ocorrencias(row))

# R3/R4: no máximo uma ocorrência da mesma disciplina por dia,
# por turma (o bloco duplo já conta como uma só ocorrência
# porque cada variável y representa o bloco inteiro).
for t in dados.turmas:
    for idx, row in disc.iterrows():
        for dia in DIAS:
            vars_dia = [
                y[(t, idx, dia, p)] for p in PERIODOS if (t, idx, dia,
p) in y
            ]
            if vars_dia:
                model.Add(sum(vars_dia) <= 1)

# R1: turma não pode ter duas aulas em simultâneo.
for t in dados.turmas:
    for dia in DIAS:
        for p in PERIODOS:
            model.Add(sum(ocupa(t, idx, dia, p) for idx in disc.index
<= 1)

# R5: professor não pode dar duas aulas em simultâneo.
# De caminho guardamos prof_occ (variável booleana "professor
# tem aula neste período"), reaproveitada no objetivo 01.
prof_occ = {}
for prof in disc["professor"].unique():
    idxs_prof = disc.index[disc["professor"] == prof]
    for dia in DIAS:
        for p in PERIODOS:
            expr = sum(
                ocupa(t, idx, dia, p)
                for t in dados.turmas
                for idx in idxs_prof
            )
            b = model.NewBoolVar(f"occ_{prof}_{dia}_{p}")
            model.Add(expr == b) # liga o booleano à soma e força <=1
(R5)

            prof_occ[(prof, dia, p)] = b

# R7: capacidade de salas (normais e especiais).
idxs_normais = disc.index[disc["sala_especial"] == ""]
for dia in DIAS:
    for p in PERIODOS:
        expr = sum(
            ocupa(t, idx, dia, p)
            for t in dados.turmas
            for idx in idxs_normais
        )
        model.Add(expr <= dados.capacidade_normal)

```

```

for tipo, cap in dados.capacidade_especial.items():
    idxs_tipo = disc.index[disc["sala_especial"] == tipo]
    for dia in DIAS:
        for p in PERIODOS:
            expr = sum(
                ocupa(t, idx, dia, p)
                for t in dados.turmas
                for idx in idxs_tipo
            )
            model.Add(expr <= cap)

return model, y, prof_occ

```

Objetivo O1 (minimizar buracos)

```

In [ ]: # [LLM] Codificação do "buraco" gerada com apoio do LLM: um período
# k é buraco de um professor num dia se há aula antes de k, aula
# depois de k, e não há aula em k. `antes`/`depois` usam
# AddMaxEquality (equivalente a um OR sobre booleanos); a variável
# `buraco` é forçada a 1 nesse caso através de um minorante linear
# (a técnica clássica para uma conjunção AND em programação linear:
# buraco >= antes + depois - ocupa_k - 1), suficiente porque o
# objetivo minimiza a soma dos buracos.
def adicionar_objetivo_buracos(model, dados, prof_occ):
    buracos = []
    for prof in dados.disciplinas["professor"].unique():
        for dia in DIAS:
            ocorre = [prof_occ[(prof, dia, p)] for p in PERIODOS]
            for k in range(2, len(PERIODOS)): # períodos "no meio": 2,3,4
                antes = model.NewBoolVar(f"antes_{prof}_{dia}_{k}")
                depois = model.NewBoolVar(f"depois_{prof}_{dia}_{k}")
                model.AddMaxEquality(antes, ocorre[:k - 1])
                model.AddMaxEquality(depois, ocorre[k:])
                buraco = model.NewBoolVar(f"buraco_{prof}_{dia}_{k}")
                model.Add(buraco >= antes + depois - ocorre[k - 1] - 1)
                buracos.append(buraco)
    model.Minimize(sum(buracos))
    return buracos

```

```

In [ ]: # [LLM] Tradução dos nomes de estado do CP-SAT – a biblioteca
# OR-Tools só os devolve em inglês (StatusName), mas o relatório
# deve estar em português.
_ESTADOS_PT = {
    "OPTIMAL": "Ótimo",
    "FEASIBLE": "Viável (não necessariamente ótimo)",
    "INFEASIBLE": "Inviável",
    "UNKNOWN": "Desconhecido (sem solução dentro do tempo limite)",
    "MODEL_INVALID": "Modelo inválido",
}

def estado_pt(solver, status):
    nome = solver.StatusName(status)
    return _ESTADOS_PT.get(nome, nome)

```

Resolver e extrair o horário

```
In [ ]: # [LLM] Gerado com apoio do LLM.
def resolver(model, limite_segundos=30):
    solver = cp_model.CpSolver()
    solver.parameters.max_time_in_seconds = limite_segundos
    solver.parameters.num_search_workers = 8
    status = solver.Solve(model)
    return solver, status

In [ ]: # [LLM] Gerado com apoio do LLM.
def extrair_horario(dados, y, solver):
    """Converte a solução do solver numa tabela (1 linha por aula ou
    bloco duplo)."""
    linhas = []
    disc = dados.disciplinas
    for (t, idx, dia, p), var in y.items():
        if solver.Value(var):
            row = disc.loc[idx]
            linhas.append({
                "turma": t,
                "disciplina": row.disciplina,
                "professor": row.professor,
                "dia": dia,
                "periodo_inicio": p,
                "duracao": 2 if row.duplo_periodo else 1,
                "sala_tipo": row.sala_especial if row.sala_especial else
                "normal",
            })
    colunas = ["turma", "disciplina", "professor", "dia", "periodo_inicio",
                "duracao", "sala_tipo"]
    if not linhas:
        return pd.DataFrame(columns=colunas)
    df = pd.DataFrame(linhas)
    ordem_dia = {d: i for i, d in enumerate(["Seg", "Ter", "Qua", "Qui",
        "Sex"])}
    df["_ordem_dia"] = df["dia"].map(ordem_dia)
    df = df.sort_values(["turma", "_ordem_dia",
        "periodo_inicio"]).drop(columns="_ordem_dia")
    return df.reset_index(drop=True)

In [ ]: modelo, y_vars, prof_occ = construir_modelo(dados)
adicionar_objetivo_buracos(modelo, dados, prof_occ)
solver, status = resolver(modelo, limite_segundos=30)

if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
    horario = extrair_horario(dados, y_vars, solver)
else:
    horario = None
    mo.md(f"**Estado do solver:** {estado_pt(solver, status)} – sem
    solução.")

mo.vstack([
    mo.md(
```

```

f"""Estado do solver:** {estado_pt(solver, status)} \n"
f"""Nº total de buracos (01):** {solver.ObjectiveValue() if horario
is not None else 'N/A'} \n"
f"""Tempo de resolução:** {solver.WallTime():.2f} s"
),
mo.ui.table(horario) if horario is not None else mo.md("_sem horário
para mostrar_"),
])

```

Estado do solver: Ótimo

Nº total de buracos (01): 0.0

Tempo de resolução: 0.04 s

Q Search...							
☐	turma str unique: 2	disciplina str unique: 6	professor str unique: 5	dia str unique: 5	periodo_inicio int64	duracao int64	sala_tipo str unique: 3
☐	7ªA	Inglês	Prof. Diana	Seg		1	1 normal
☐	7ªA	Matemática	Prof. Ana	Seg		2	1 normal
☐	7ªA	História	Prof. Diana	Seg		3	1 normal
☐	7ªA	Educação Física	Prof. Eduardo	Seg		4	2 Ginásio
☐	7ªA	Português	Prof. Bruno	Ter		2	1 normal
☐	7ªA	História	Prof. Diana	Ter		3	1 normal
☐	7ªA	Inglês	Prof. Diana	Ter		4	1 normal
☐	7ªA	Matemática	Prof. Ana	Ter		5	1 normal
☐	7ªA	Português	Prof. Bruno	Qua		2	1 normal
☐	7ªA	Ciências	Prof. Carla	Qua		3	1 Laboratório

32 rows, 7 columns No selection

3. Verificação automática (R1-R8)

Função independente do solver que confere se um horário (no formato produzido por `extrair_horario`) respeita R1-R8. Serve tanto para validar o resultado do CP-SAT como, mais tarde, o resultado da construção incremental (secção seguinte).

```

In [ ]: # [LLM] Gerado com apoio do LLM e testado com dois horários
# fabricados à mão (um propositadamente inválido, cobrindo R1--R6,
# e um quase-válido) antes de ser usado sobre a solução do solver –
# ver diálogo linkado no topo do notebook.
def validar_horario(df, dados):
    """Devolve lista de strings com violações (vazia = válido)."""
    problemas = []
    disc = dados.disciplinas.set_index("disciplina")

    def periodos_ocupados(row):
        if row.duracao == 1:
            return [row.periodo_inicio]
        return [row.periodo_inicio, row.periodo_inicio + 1]

    ocupacao_turma, ocupacao_prof, ocupacao_sala = {}, {}, {}
    contagem = {}

    for _, row in df.iterrows():
        prof = disc.loc[row.disciplina, "professor"]
        duplo = bool(disc.loc[row.disciplina, "duplo_periodo"])

```

```

sala_esp = disc.loc[row.disciplina, "sala_especial"]
tipo_sala = sala_esp if sala_esp else "normal"

if duplo and row.duracao != 2:
    problemas.append(f"R4: {row.disciplina}/{row.turma} devia ser
bloco duplo")
if (not duplo) and row.duracao != 1:
    problemas.append(f"R4: {row.disciplina}/{row.turma} não é duplo
mas duracao={row.duracao}")

for p in periodos_ocupados(row):
    ocupacao_turma.setdefault((row.turma, row.dia, p),
[]).append(row.disciplina)
    ocupacao_prof.setdefault((prof, row.dia, p),
[]).append((row.turma, row.disciplina))
    ocupacao_sala[(tipo_sala, row.dia, p)] =
ocupacao_sala.get((tipo_sala, row.dia, p), 0) + 1
    if (prof, row.dia, p) in dados.indisponibilidades:
        problemas.append(f"R6: {prof} dá aula em {row.dia} P{p} mas
está indisponível")

chave = (row.turma, row.disciplina)
contagem[chave] = contagem.get(chave, 0) +
len(periodos_ocupados(row))

linhas_no_dia = df[
    (df.turma == row.turma) & (df.disciplina == row.disciplina) &
(df.dia == row.dia)
]
if len(linhas_no_dia) > 1:
    problemas.append(f"R3: {row.disciplina}/{row.turma} tem >1
ocorrência em {row.dia}")

for k, v in ocupacao_turma.items():
    if len(v) > 1:
        problemas.append(f"R1: turma {k[0]} tem {v} em simultâneo em
{k[1]} P{k[2]}")
    for k, v in ocupacao_prof.items():
        if len(v) > 1:
            problemas.append(f"R5: professor tem {v} em simultâneo em
{k[1]} P{k[2]} ({k[0]})")
    for k, v in ocupacao_sala.items():
        tipo = k[0]
        cap = dados.capacidade_normal if tipo == "normal" else
dados.capacidade_especial.get(tipo, 0)
        if v > cap:
            problemas.append(f"R7: sala tipo {tipo} tem {v} aulas em {k[1]}
P{k[2]}, capacidade={cap}")

for (turma, disciplina), carga in contagem.items():
    esperado = int(disc.loc[disciplina, "carga_semanal"])
    if carga != esperado:
        problemas.append(f"R2: {disciplina}/{turma} tem carga {carga},
esperado {esperado}")

return problemas

```

```
In [ ]: if horario is not None:
        problemas = validar_horario(horario, dados)
        if problemas:
            resultado = mo.callout(mo.md("\n".join(f"- {p}" for p in
problemas))), kind="danger")
        else:
            resultado = mo.callout(mo.md("Horário válido: nenhuma violação de
R1-R8 encontrada."), kind="success")
        else:
            resultado = mo.md("_sem horário para validar_")
        resultado
```

Horário válido: nenhuma violação de R1-R8 encontrada.

4. Construção incremental (R9)

Abordagem escolhida: dado H_0 (horário anterior) e os novos dados, identificamos que professores tiveram a disponibilidade alterada. Para os professores **não afetados**, fixamos as suas variáveis y exatamente aos valores de H_0 (`model.Add(var == valor)`) — isto reduz drasticamente o espaço de procura, porque o CP-SAT só tem decisões reais a tomar para o(s) professor(es) afetado(s). Sobre essas variáveis "livres", adicionamos um objetivo que penaliza desligar uma aula que estava ativa em H_0 e continua possível — ou seja, **minimizar o número de aulas alteradas** (a única coisa que R9 pede a H_1 ; não tem de ser ótimo em O1).

Isto combina duas das técnicas sugeridas no enunciado: reaproveitar H_0 como ponto de partida e resolver apenas o subproblema afetado.

```
In [ ]: # [LLM] Gerado com apoio do LLM; testado com dados/ → dados_v2/
# reais (a mudança de disponibilidade da Prof. Ana) – ver diálogo.
def identificar_professores_afetados(dados_antigo, dados_novo):
    """Professores cuja disponibilidade mudou entre duas versões
    dos dados (comparando os conjuntos de exceções)."""
    diff = dados_antigo.indisponibilidades.symmetric_difference(
        dados_novo.indisponibilidades
    )
    return {professor for (professor, dia, periodo) in diff}
```

```
In [ ]: # [LLM] Gerado com apoio do LLM; testado com o mesmo caso real.
def fixar_nao_afetados(model, y, dados_novo, horario_antigo,
    professores_afetados):
    """Fixa (model.Add(var == valor)) todas as variáveis y de
    professores NÃO afetados aos valores de horario_antigo.
    Devolve a lista de chaves que ficaram livres (as do(s)
    professor(es) afetado(s))."""
    disc = dados_novo.disciplinas
    ocupados_antigos = set(
        (row.turma, row.disciplina, row.dia, row.periodo_inicio)
```

```

        for row in horario_antigo.itertuples()
    )
    chaves_livres = []
    for (t, idx, dia, p), var in y.items():
        prof = disc.loc[idx, "professor"]
        if prof in professores_afetados:
            chaves_livres.append((t, idx, dia, p))
            continue
        disciplina = disc.loc[idx, "disciplina"]
        valor = 1 if (t, disciplina, dia, p) in ocupados_antigos else 0
        model.Add(var == valor)
    return chaves_livres

```

```

In [ ]: # [LLM] Gerado com apoio do LLM; testado com o mesmo caso real –
# confirmou-se que das ocorrências antigas do professor afetado só
# entram no objetivo as que ainda são fisicamente possíveis (as
# que deixaram de o ser, por já não existir a variável, resultam
# em mudança forçada, sem custo adicional a otimizar).
def objetivo_minimizar_mudancas(model, y, dados_novo, horario_antigo,
chaves_livres):
    """Adiciona a H1 o objetivo de minimizar o nº de aulas (entre as
    variáveis livres) que deixam de estar onde estavam em H0."""
    disc = dados_novo.disciplinas
    ocupados_antigos = set(
        (row.turma, row.disciplina, row.dia, row.periodo_inicio)
        for row in horario_antigo.itertuples()
    )
    termos = []
    for (t, idx, dia, p) in chaves_livres:
        disciplina = disc.loc[idx, "disciplina"]
        if (t, disciplina, dia, p) in ocupados_antigos:
            termos.append(1 - y[(t, idx, dia, p)])
    model.Minimize(sum(termos) if termos else 0)
    return termos

```

```

In [ ]: # [LLM] Junta as três funções anteriores com construir_modelo
# (reaproveitado tal e qual, já que a nova disponibilidade já é
# respeitada automaticamente por inicios_permitidos).
def construir_modelo_incremental(dados_antigo, dados_novo, horario_antigo):
    professores_afetados = identificar_professores_afetados(dados_antigo,
dados_novo)
    model, y, prof_occ = construir_modelo(dados_novo)
    chaves_livres = fixar_nao_afetados(model, y, dados_novo,
horario_antigo, professores_afetados)
    objetivo_minimizar_mudancas(model, y, dados_novo, horario_antigo,
chaves_livres)
    return model, y, professores_afetados

```

```

In [ ]: # [LLM] Gerado com apoio do LLM.
def contar_mudancas(horario_antigo, horario_novo):
    """Nº de aulas que existiam em horario_antigo num certo
    (turma, disciplina, dia, periodo_inicio) e deixaram de lá
    estar em horario_novo (assume o mesmo nº total de aulas nos
    dois, o que se verifica sempre que a carga semanal não muda)."""
    def chaves(h):

```

```

        return set(zip(h.turma, h.disciplina, h.dia, h.periodo_inicio))
    return len(chaves(horario_antigo) - chaves(horario_novo))

```

Comparação: incremental vs. resolver H1 do zero

```

In [ ]: # Independente do dropdown da secção 1 – carrega sempre os dois
# conjuntos de dados explicitamente, para a comparação ser sempre
# reproduzível.
dados_h0_src = carregar_dados("dados")
dados_h1_src = carregar_dados("dados_v2")

```

```

In [ ]: # H0: horário inicial, a partir de dados/.
_modelo_h0, _y_h0, _prof_occ_h0 = construir_modelo(dados_h0_src)
adicionar_objetivo_buracos(_modelo_h0, dados_h0_src, _prof_occ_h0)
_solver_h0, _status_h0 = resolver(_modelo_h0, limite_segundos=30)
horario_h0 = extrair_horario(dados_h0_src, _y_h0, _solver_h0)

```

```

In [ ]: # H1 resolvido do ZERO (não reaproveita H0), para comparação.
_inicio = time.perf_counter()
_modelo_zero, _y_zero, _prof_occ_zero = construir_modelo(dados_h1_src)
adicionar_objetivo_buracos(_modelo_zero, dados_h1_src, _prof_occ_zero)
solver_zero, status_zero = resolver(_modelo_zero, limite_segundos=30)
tempo_zero = time.perf_counter() - _inicio
horario_zero = extrair_horario(dados_h1_src, _y_zero, solver_zero)

```

```

In [ ]: # H1 resolvido de forma INCREMENTAL (reaproveita H0).
_inicio = time.perf_counter()
_modelo_inc, _y_inc, profs_afetados = construir_modelo_incremental(
    dados_h0_src, dados_h1_src, horario_h0
)
solver_inc, status_inc = resolver(_modelo_inc, limite_segundos=30)
tempo_inc = time.perf_counter() - _inicio
horario_inc = extrair_horario(dados_h1_src, _y_inc, solver_inc)

```

```

In [ ]: _mudancas_zero = contar_mudancas(horario_h0, horario_zero)
_mudancas_inc = contar_mudancas(horario_h0, horario_inc)

mo.md(
    f"***Professor(es) afetado(s) pela mudança:** {'',
    '.join(profs_afetados)}\n\n"
    f"| Abordagem | Estado | Tempo (s) | Aulas alteradas vs. H0 |\n"
    f"| ---|---|---|---|\n"
    f"| Do zero | {estado_pt(solver_zero, status_zero)} | {tempo_zero:.3f}
| {_mudancas_zero} |\n"
    f"| Incremental | {estado_pt(solver_inc, status_inc)} | {tempo_inc:.3f}
| {_mudancas_inc} |\n"
)

```

Professor(es) afetado(s) pela mudança: Prof. Ana

Abordagem	Estado	Tempo (s)	Aulas alteradas vs. H0
Do zero	Ótimo	0.117	27
Incremental	Ótimo	0.087	0

5. Teste com dados adicionais (não fornecidos)

Esta secção corre sempre com `dados_extra/` explicitamente (independente da escolha no dropdown da secção 1), para deixar registado — mesmo num export estático — que o sistema funciona sem alterações de código com uma turma e disciplinas extra (incluindo duas disciplinas a partilhar o mesmo tipo de sala especial).

```
In [ ]: dados_extra_fixo = carregar_dados("dados_extra")
_modelo_extra, _y_extra, _prof_occ_extra =
construir_modelo(dados_extra_fixo)
adicionar_objetivo_buracos(_modelo_extra, dados_extra_fixo,
_prof_occ_extra)
_solver_extra, _status_extra = resolver(_modelo_extra, limite_segundos=30)
horario_extra_fixo = extrair_horario(dados_extra_fixo, _y_extra,
_solver_extra)
_problemas_extra = validar_horario(horario_extra_fixo, dados_extra_fixo)

mo.vstack([
    mo.md(
        f"""Turmas:** {' '.join(dados_extra_fixo.turmas)} "
        f"({len(dados_extra_fixo.turmas)} turmas, "
        f"{len(dados_extra_fixo.disciplinas)} disciplinas) \n"
        f"""Estado do solver:** {estado_pt(_solver_extra, _status_extra)}
\n"
        f"""Nº total de buracos (01):** {_solver_extra.ObjectiveValue()}
\n"
        f"""Tempo de resolução:** {_solver_extra.WallTime():.2f} s \n"
        f"""Validação R1-R8:** "
        + ("□ sem violações" if not _problemas_extra else f"x
{len(_problemas_extra)} violações: {_problemas_extra}")
    ),
    mo.ui.table(horario_extra_fixo, label="Horário gerado (dados_extra)"),
])
```

